"Linux Command Execution Time Profiler"

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

By

PARAM MANASVI 2520030072

BURLA VIDA SWETHA 2520030100

Under the Esteemed Guidance of

Dr. K. Hema

Assistant Professor

Department of Computer Science and Engineering



K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING & DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

Declaration

The Project Report entitled "Linux Command Execution Time Profiler" is a record of Bonafide work of BURLA VIDA SWETHA - 2520030100, PARAM MANASVI - 2520030072 submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

PARAM MANASVI 2520030072

BURLA VIDA SWETHA 2520030100

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is to certify that the mini project based report entitled "Linux Command Execution Time Profiler" is a bonafide work done and submitted by BURLA VIDA SWETHA - 2520030100, PARAM MANASVI - 2520030072, in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year 2024-2025.

Signature of the Guide **Signature of the Course Coordinator** **Signature of the HOD**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator and my guide Dr. K. Hema, Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to Dr. P Venkateshwara Roa (CSE), Head of the Department for Computer Science Engineering for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

PARAM MANASVI 2520030072

BURLA VIDA SWETHA 2520030100

Abstract

When a command runs on Linux, the user normally sees only its output and has no idea how much CPU time, memory or kernel activity it needed. Existing tools such as time, top and strace give this information, but they are separate command-line programs and are not easy for beginners to read or compare. Our project, the Design and Development of a Linux Command Execution Time Profiler, brings these measurements together in one web dashboard that runs on the local machine.

The system is written in C and runs entirely in Linux user space. A small hand-written C HTTP server receives a request from the browser, checks the command against a list of 20 approved commands and creates a child process using fork() and execvp(), never using system() or a shell. The parent waits with wait4(), which returns the resource usage of the child: user CPU time, system CPU time, peak memory (max RSS), page faults and context switches. Wall-clock time is measured with clock_gettime(CLOCK_MONOTONIC), the output of the command is captured through pipe() and dup2(), and a command that exceeds its time limit is stopped with SIGTERM followed by SIGKILL on its process group.

Additional features include repeated runs with min, mean, max and standard deviation, comparison of two commands, system-call tracing with ptrace(), live process monitoring through /proc, and a SQLite database in WAL mode that stores the history and statistics of every run. In summary, the project gives a simple, safe and visual way to understand how Linux commands use system resources, and demonstrates the main concepts of operating systems and systems programming.

Index

S.No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		4
	Abstract		5
1	Introduction	1.1 Background; 1.2 Problem Statement; 1.3 Scope; 1.4 Objectives; 1.5 Importance; 1.6 Target Users	7
2	OS Concepts and Background Study	2.1 OS Concepts; 2.2 Existing Methods; 2.3 Proposed Approach; 2.4 Advantages	9
3	System Requirements	3.1 Hardware; 3.2 Software; 3.3 OS Environment; 3.4 Development Tools	11
4	Technology and Tools Used	4.1 C Programming; 4.2 GCC; 4.3 System Calls; 4.4 Synchronization; 4.5 Linux Commands; 4.6 Other Tools	12
5	System Architecture and Design	5.1 Architecture; 5.2 Workflow; 5.3 Data Flow; 5.4 Modules; 5.5 Input and Output Design	13
6	Implementation	6.1 Module-wise; 6.2 Algorithm; 6.3 Core C Code; 6.4 System Calls; 6.5 Process Implementation; 6.6 IPC; 6.7 Memory; 6.8 User Interface	15
7	Execution and Demonstration	7.1 Compilation; 7.2 Execution; 7.3 Sample Inputs/Outputs; 7.4 Screenshots; 7.5 OS Concepts Demonstrated	17
8	Conclusion	8.1 Summary; 8.2 Objectives Achieved; 8.3 Results; 8.4 Skills and Learning	19
9	References	9.1 Books; 9.2 Research Papers; 9.3 Linux Documentation; 9.4 Online Resources	20

CHAPTER 1 INTRODUCTION

1.1 Background of the Project

Every command typed in a Linux terminal becomes a process. The kernel creates it, schedules it on a CPU, gives it memory, and collects its resource usage when it ends. A command such as ls may finish in a few milliseconds, while gzip on a large file may use much more CPU time and memory. Knowing how long a command takes and what resources it uses is important for understanding performance.

This project, the Linux Command Execution Time Profiler, was built for the Operating Systems and Systems Programming scenario: run a Linux command, measure its execution time and resource usage, and present the results clearly. The system is delivered as a small web application so that the behaviour of the operating system can be seen as well as read about. The whole back end, including the web server, is written in C and runs in Linux user space. No kernel modification is required, and the only external library is SQLite.

1.2 Problem Statement

Existing tools such as time, top, ps and strace each show only a part of the picture, produce plain text, and cannot store or compare results. Running arbitrary commands from a web page is also dangerous if a shell is used. A better solution must satisfy the following requirements:

- run a chosen command and measure its wall-clock, user and system CPU time accurately;
- report memory use, page faults and context switches of the command;
- capture the output and errors of the command and stop it if it runs too long;
- allow only safe commands and reject shell operators such as ; | & > <;
- store every run so that results can be compared and averaged; and
- show the results in a simple web interface.

The problem addressed is therefore: to design and implement, using only standard Linux system calls, a profiler that is accurate (kernel resource data), safe (allowlist, no shell), observable (syscall trace, /proc monitor) and persistent (SQLite history) through a web dashboard.

1.3 Scope of the Project

- Command execution: fork(), execvp() and wait4() for 20 allowlisted commands.
- Measurements: wall time, user/system CPU time, max RSS, page faults, context switches, exit status.
- Analysis: repeated runs (2 to 20) with statistics, and side-by-side comparison of two commands.
- Tracing and monitoring: ptrace() system-call summary and /proc/<pid> inspection.
- Persistence: history of the latest 100 runs and per-command statistics in SQLite.

Outside the scope are kernel modification, arbitrary shell commands, user accounts, HTTPS, and access from other machines, since the server binds to 127.0.0.1 only.

1.4 Objectives of the Project

- To apply process creation (fork, exec, wait) in a working application.
- To collect resource usage of a child process using wait4() and struct rusage.
- To capture output using pipes and enforce time limits using signals.
- To trace system calls with ptrace and read process data from /proc.
- To use SQLite as the persistence layer and build a REST-style HTTP server in C.
- To provide a browser front end using HTML, CSS and JavaScript.

1.5 Importance of the Application

For a user, the profiler is a safe and easy way to find out which command is faster or lighter. For a student of operating systems, it connects theory with practice: concepts such as fork/exec, zombie reaping, user and kernel time, page faults, context switches and system calls can be observed on a real system. The code base is small (about 1,800 lines of C) and commented, so it can be read in full.

1.6 Target Users / Audience

Audience	How they use the system
Students of Operating Systems	Study process creation, resource usage and system calls on a real Linux machine.
Faculty and evaluators	Verify the use of system calls and safe process handling during project review.
General Linux users	Compare the speed and memory use of commands.
Systems programmers	Use the code as a reference for fork/exec, wait4 and ptrace.

CHAPTER 2 OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

Process creation: a process is a program in execution with its own PID and address space. fork() duplicates the calling process and execvp() replaces the child with the requested program. The server forks once for each profiling request.

Process states and scheduling: while a command runs, the scheduler moves it between ready, running and waiting states. Voluntary context switches occur when the process blocks (for example waiting for I/O); involuntary ones occur when the scheduler preempts it. Both are reported by the profiler.

CPU time: user time is spent running program code; system time is spent inside the kernel on behalf of the program. Wall time is the real elapsed time and includes waiting.

Memory and page faults: a minor page fault is resolved without disk access; a major page fault needs disk I/O. The peak resident set size (max RSS) shows the highest physical memory used.

Signals: SIGTERM asks a process to end, SIGKILL forces it, and SIGCHLD tells the parent that a child has finished. Pipes carry the output of the child to the parent, and ptrace lets a parent observe every system call of a child. The /proc filesystem exposes live process information.

Table 2.1: Mapping of OS concepts to implementation

OS concept	System call / interface	Where used
Process creation	fork(), execvp()	profiler.c
Waiting + resource usage	wait4(), struct rusage	profiler.c
Timing	clock_gettime(CLOCK_MONOTONIC)	profiler.c
Output capture	pipe(), dup2(), fcntl()	profiler.c
Process groups / timeout	setpgid(), kill(-pgid)	profiler.c
Zombie reaping	sigaction(), SIGCHLD	server.c
Syscall tracing	ptrace(PTRACE_SYSCALL)	syscall_trace.c
Process inspection	/proc/<pid>/status, stat	server.c
IPC / persistence	SQLite in WAL mode	db.c
Networking	socket(), bind(), listen(), accept()	server.c, http.c

2.2 Existing Methods / Algorithms

Table 2.2: Comparison of existing methods

Method	How it works	Limitations
time command	Prints real, user, sys time.	No memory or fault data in default form; no history.
top / htop	Live view of CPU and memory.	Not per-command; hard for short commands.
strace	Traces system calls.	Heavy text output; no web view or comparison.

Method	How it works	Limitations
perf	Hardware counters and profiling.	Complex; needs privileges.
Shell scripts	Wrap commands with date.	Low accuracy; unsafe if shell is used.

2.3 Proposed Approach

- The browser sends the command, arguments and timeout to the server as JSON (POST /api/run).
- The server checks the command against the allowlist and rejects shell metacharacters.
- It creates pipes and calls fork(); the child sets its own process group, redirects stdout and stderr, and calls execvp().
- The parent records the start time, drains the pipes without blocking and checks the timeout.
- On exit, wait4() returns the exit status and resource usage; the parent records the end time.
- The result is saved in SQLite and returned to the browser as JSON for display.

2.4 Advantages of the Proposed Approach

- Accurate: values come directly from the kernel (rusage) and a monotonic clock.
- Safe: allowlist, no shell, no path traversal, localhost only.
- Controlled: timeouts stop runaway commands, including their child processes.
- Clean life cycle: children are always reaped, so no zombies remain.
- Persistent and comparable: all runs are stored with min/avg/max statistics.
- Small dependency footprint: only C, Linux system calls and SQLite.

CHAPTER 3 SYSTEM REQUIREMENTS

3.1 Hardware Requirements

Table 3.1: Hardware requirements

Component	Minimum	Recommended
Processor	1 core, x86-64	2 or more cores
Memory (RAM)	512 MB	2 GB or more
Disk space	50 MB free	500 MB free
Display	Any screen that runs a browser	1280 x 800 or larger
Network	Loopback (localhost) only	Loopback only

3.2 Software Requirements

Table 3.2: Software requirements

Software	Version / package	Purpose
Operating system	Ubuntu 22.04 or later (tested on 24.04), or Ubuntu on WSL2	Linux system calls and /proc
C compiler	gcc (build-essential), -std=c11	Compiles the source
Build tool	make	Builds using the Makefile
SQLite library	libsqlite3-dev	Database engine and sqlite3.h
Web browser	Chrome, Firefox or Edge	Runs the dashboard
Utilities	curl	Testing the REST API

3.3 Operating System Environment - Ubuntu / WSL

The system uses Linux-specific features (/proc, wait4, ptrace), so it must run on Linux. It runs natively on Ubuntu or Debian and inside Windows Subsystem for Linux (WSL2). Prepare the environment with:

```
sudo apt update
sudo apt install -y build-essential libsqlite3-dev curl
```

Listing 3.1: Installing the required packages

Note: ptrace may be restricted in some WSL setups. In that case the Syscall Trace tab reports an error, and all other features still work.

3.4 Development Tools and Compilers

Table 3.3: Development tools

Tool	Use in the project
GNU C Compiler (gcc)	Compiles the .c files and links the profiler_server executable.
GNU Make	Automates the build; targets all, run, clean and test.
gdb	Source-level debugging (the build uses -g).
curl	Calls the REST API from the terminal.
Text editor / IDE	Visual Studio Code for C, HTML, CSS and JavaScript.

The compiler options in the Makefile are -Wall -Wextra -Wpedantic -std=c11 -D_GNU_SOURCE -Iinclude -O2 -g, and the link step adds -lsqlite3.

CHAPTER 4 TECHNOLOGY AND TOOLS USED

4.1 C Programming

The back end is written in C (C11) because it gives direct access to the Linux system calls this project is about. The source is organised as five implementation files and four headers.

Table 4.1: Source files and size

File	Lines	Responsibility
src/server.c	530	main(), TCP listener, request routing, SIGCHLD reaper, /proc monitor
src/profiler.c	281	fork/exec, pipes, wait4, timeout, allowlist
src/syscall_trace.c	348	ptrace-based system-call counting and timing
src/db.c	261	SQLite schema, inserts, history and statistics in JSON
src/http.c	170	HTTP request reader, response writer, static files
include/*.h	4 files	db.h, http.h, profiler.h, syscall_trace.h
public/	3 files	index.html, style.css, app.js (dashboard)

4.2 GCC Compiler

GCC compiles each source file to an object file and links them with the SQLite library. Strict warning flags are enabled and the project builds without warnings on Ubuntu 24.04.

4.3 Linux System Calls

The main system calls are fork(), execvp(), wait4(), pipe(), dup2(), fcntl(), setpgid(), kill(), sigaction(), clock_gettime(), ptrace(), and the socket calls socket(), bind(), listen() and accept().

4.4 POSIX Threads / Synchronization

The server handles one request at a time and does not use threads. Synchronization is achieved with wait4() (the parent waits for the child), a non-blocking drain loop on the pipes, and SQLite's WAL mode with a busy time-out for safe database access.

4.5 Linux Commands and Utilities

The allowlist contains 20 commands: ls, pwd, echo, cat, head, tail, wc, sort, uniq, grep, find, sleep, gzip, date, whoami, uname, df, du, uptime and free. curl is used for API testing, and ps and /proc for process inspection.

4.6 Other Tools / Libraries Used

- SQLite 3 for storing runs and computing statistics (min, avg, max, standard deviation) in SQL.
- HTML5, CSS3 and vanilla JavaScript with canvas charts for the front end; no frameworks.
- Shell scripts tests/smoke_test.sh and tests/scenario_test.sh for automated testing.

CHAPTER 5 SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The system has three layers. The browser dashboard sends JSON requests to the C HTTP server on 127.0.0.1. The server routes each request to the profiler, the syscall tracer, the /proc monitor or the database module. Results go back to the browser as JSON.

Table 5.1: Layers of the system

Layer	Component	Role
Presentation	public/index.html, app.js, style.css	Forms, tables and charts
Server	server.c, http.c	Listener, routing, JSON responses
Profiling	profiler.c, syscall_trace.c	Run, measure and trace commands
Storage	db.c, SQLite file (WAL)	History and statistics
Kernel	fork, wait4, ptrace, /proc	Process and resource services

5.2 System Workflow

- The user selects a command, enters arguments and a timeout, and clicks Run.
- The server validates the command, arguments and timeout (100 to 60000 ms).
- The profiler forks a child, which executes the command; the parent measures and waits.
- Metrics and output are saved in the runs table of SQLite.
- The dashboard shows the metrics, output and a time-breakdown chart.

5.3 Flowchart / Data Flow

```
Browser --POST /api/run--> server.c --> command_is_allowed()?
                                    | no  -> HTTP 403 / 400 error
                                    | yes
                                    v
                         pipe() -> fork() -> child: setpgid, dup2, execvp
                                    |
                   parent: clock_gettime, drain pipes, check timeout
                                    |
                   wait4() -> status + rusage -> clock_gettime
                                    |
                   db_add_run() -> JSON response -> Browser
```

Figure 5.1: Data flow of one profiling run

5.4 Module Description

Table 5.2: Modules

Module	Description
Run	Executes one command and shows all metrics, output and time chart.
Repeat	Runs the same command N times (2 to 20) and shows min, mean, max, stddev.
Compare	Runs two commands with the same timeout and shows side-by-side results.
Syscall Trace	Uses ptrace to list total syscalls and the top 10 by count with elapsed time.
Monitor PID	Reads /proc/<pid>/status and stat; live RSS timeline polled every second.

Module	Description
History	Shows the latest 100 runs from SQLite.
Stats	Per-command min/avg/max wall time, average CPU and RSS with a bar chart.

5.5 Input and Output Design

Input is a JSON object: {"command":"ls","args":"-la","timeout_ms":5000}. Output is a JSON object containing status, wall_ms, user_ms, system_ms, max_rss_kb, minor_faults, major_faults, voluntary_cs, involuntary_cs, exit_code, term_signal, timed_out, stdout and stderr. The status is completed, failed, timed_out or signal_terminated.

Table 5.3: REST API

Method	Endpoint	Purpose
GET	/api/allowlist	List of runnable commands
POST	/api/run	Run and profile a command
POST	/api/trace	Syscall trace of a command
GET	/api/monitor/<pid>	Snapshot of a process from /proc
GET	/api/history	Latest 100 runs
GET	/api/stats	Per-command statistics

CHAPTER 6 IMPLEMENTATION

6.1 Module-wise Implementation

profiler.c contains profiler_run(), which validates the input, builds argv, creates the pipes, forks, measures and returns a profile_result_t structure. syscall_trace.c contains syscall_trace_run(), which runs the command under ptrace and counts system calls on x86_64. db.c opens the database in WAL mode and provides db_add_run(), db_history_json() and db_stats_json(). http.c reads requests and writes responses, and server.c routes the API calls and serves the files in public/.

6.2 Algorithm / Pseudocode

```
profiler_run(command, args, timeout):
  if command not in allowlist or timeout out of range: return error
  build argv from command and args (no shell)
  create pipes for stdout and stderr
  start = clock_gettime(MONOTONIC)
  pid = fork()
  if pid == 0: setpgid; dup2 pipes; execvp(command); _exit(127)
  parent: make pipes non-blocking
  loop until child done and pipes closed:
      if elapsed > timeout: kill(-pid, SIGTERM); wait 150 ms; kill(-pid,
SIGKILL)
      read available stdout/stderr data
      if wait4(pid, &status, WNOHANG, &rusage) returns pid: child_done
  end = clock_gettime(MONOTONIC)
  fill result from rusage, status and elapsed time
```

6.3 Core C Implementation

```
pid_t pid = fork();
if (pid == 0) {
    setpgid(0, 0);
    dup2(po[1], STDOUT_FILENO);
    dup2(pe[1], STDERR_FILENO);
    close(po[0]); close(po[1]); close(pe[0]); close(pe[1]);
    execvp(command, argv);
    dprintf(STDERR_FILENO, "execvp failed: %s\n", strerror(errno));
    _exit(127);
}
setpgid(pid, pid);
close(po[1]); close(pe[1]);
```

Listing 6.1: Creating the child process (profiler.c)

```
if (!child_done && !r->timed_out
    && timespec_ms(wall_start, now) > timeout_ms) {
    r->timed_out = 1;
    kill(-pid, SIGTERM);
    usleep(150000);
    kill(-pid, SIGKILL);
}
```

Listing 6.2: Timeout handling (profiler.c)

6.4 Linux System Calls / APIs Used

Table 6.1: System calls and purpose

System call	Purpose
fork()	Create the child process for each command

System call	Purpose
execvp()	Run the command without a shell
wait4()	Reap the child and obtain exit status and resource usage
pipe(), dup2()	Capture stdout and stderr
fcntl(O_NONBLOCK)	Non-blocking reads so the timeout can be checked
setpgid(), kill(-pgid)	Terminate the whole process group on timeout
clock_gettime()	High-resolution wall-clock timing
sigaction(SIGCHLD)	Zombie-free reaping, SIGPIPE ignored
ptrace()	Observe system calls of the child
socket(), bind(), listen(), accept()	HTTP server on 127.0.0.1

6.5 Process / Thread Implementation

Each profiling request creates one child process. The child is placed in its own process group so that a timeout can terminate it and any processes it started. The server is single-threaded; the parent never blocks forever because the pipes are non-blocking and the timeout is checked in every loop iteration. A SIGCHLD handler with waitpid(-1, NULL, WNOHANG) removes any leftover children, so no zombie processes remain.

6.6 IPC / Synchronization Implementation

Pipes carry output from child to parent. The exit status and resource usage return through wait4(). The database is the persistence layer: it runs in WAL mode so readers (history, statistics) are not blocked by writers. Statistics such as min, average, maximum and standard deviation are computed inside SQL queries.

6.7 Memory Management Implementation

The captured output is stored in heap buffers (out and err in profile_result_t) that grow as needed and are released by profiler_result_free(). Argument strings are copied with strdup() and freed after execution. Fixed-size buffers with snprintf() prevent overflows. The reported max RSS (ru_maxrss, in KiB) is the peak memory of the command, not of the server.

6.8 Optional User Interface Implementation

The dashboard (public/index.html, style.css, app.js) has seven tabs: Run, Repeat, Compare, Syscall Trace, Monitor PID, History and Stats. It uses fetch() to call the API and draws bar charts and the live RSS timeline on an HTML canvas, without any external library.

CHAPTER 7 EXECUTION AND DEMONSTRATION

7.1 Compilation in Ubuntu

```
cd Linux_Command_Execution_Time_Profiler
make
# Build successful: ./profiler_server
```

Listing 7.1: Building the project

7.2 Execution Commands

```
./profiler_server          # default port 8080
./profiler_server 8090     # custom port
make test                  # run smoke and scenario tests
```

Open http://127.0.0.1:8080 in a browser. Stop the server with Ctrl+C.

7.3 Sample Inputs and Outputs

Table 7.1: Sample test cases

Input	Expected behaviour	Result
echo hello world	stdout captured, status completed	Pass
wc -l on a file	Line count shown, exit code 0	Pass
grep for missing text	Non-zero exit, status failed	Pass
sleep 10, timeout 300 ms	Stopped by SIGTERM/SIGKILL, timed_out	Pass
bash	Rejected, HTTP 403 (not allowlisted)	Pass
args containing ;	Rejected, HTTP 400	Pass
Syscall trace of ls	Total syscalls and top 10 listed	Pass (needs ptrace)

Sample output for POST /api/run with echo (from the project README):

```
{ "status":"completed", "wall_ms":2.341, "user_ms":0.873,
  "system_ms":0.411, "max_rss_kb":1024, "minor_faults":42,
  "major_faults":0, "voluntary_cs":1, "involuntary_cs":0,
  "exit_code":0, "timed_out":false, "stdout":"hello world\n" }
```

The test suite contains 10 smoke checks and 12 scenario checks that can be run with make test.

7.4 Screenshots of Execution

[Insert screenshots here: (1) Run tab with metrics, (2) Repeat tab statistics, (3) Compare tab chart, (4) Syscall Trace tab, (5) Monitor PID live chart, (6) History and Stats tabs.]

7.5 Demonstration of OS Concepts

- Process creation: ps --forest shows the server and its short-lived child while a command runs.
- Resource accounting: user and system times and context switches come from the kernel through wait4().
- Signals: a sleep command with a short timeout is terminated, showing SIGTERM and SIGKILL handling.
- Page faults: running gzip or sort on a large file raises minor faults and max RSS.
- System calls: the trace tab shows how many read, write and openat calls a command makes.

- Security: shell commands and metacharacters are refused by the allowlist.

CHAPTER 8 CONCLUSION

8.1 Summary of the Project

The Linux Command Execution Time Profiler measures how Linux commands use CPU, memory and kernel resources and presents the results in a web dashboard. It is built in C using fork, execvp, wait4, pipes, signals, ptrace and /proc, with SQLite for history and statistics.

8.2 Objectives Achieved

- Commands run safely through fork/exec without a shell.
- Wall, user and system time, memory, page faults and context switches are measured.
- Output is captured and timeouts are enforced for the whole process group.
- Repeated runs, comparison, syscall tracing and process monitoring are available.
- Results are stored in SQLite and shown in a browser dashboard.

8.3 Results Achieved

The project builds with strict warning flags on Ubuntu 24.04 and passes the smoke and scenario tests provided with it. Timeouts stop long-running commands, non-allowlisted commands and unsafe arguments are rejected, and no zombie processes remain.

8.4 Skills and Learning Outcomes

- Practical use of process, signal, pipe and timing system calls.
- Understanding of CPU time, memory and context-switch accounting.
- Writing a small HTTP server and JSON API in C.
- Using SQLite, shell testing and a plain JavaScript front end.

Future work can include a CPU-time and memory limit using setrlimit(), more commands, graph export and support for other architectures in the syscall tracer.

CHAPTER 9 REFERENCES

9.1 Books

- A. Silberschatz, P. B. Galvin and G. Gagne, Operating System Concepts, Wiley.
- W. R. Stevens and S. A. Rago, Advanced Programming in the UNIX Environment, Addison-Wesley.
- M. Kerrisk, The Linux Programming Interface, No Starch Press.

9.2 Research Papers

- Papers on process scheduling and performance profiling in Linux, as referred to in the course material.

9.3 Linux / GNU Documentation

- Linux manual pages: fork(2), execve(2), wait4(2), pipe(2), ptrace(2), getrusage(2), clock_gettime(2), proc(5).
- GNU C Library and GNU Make manuals.

9.4 Online Resources

- SQLite documentation, https://www.sqlite.org/docs.html
- man7.org Linux man pages, https://man7.org/linux/man-pages/